# Chapter 14 — Browser APIs and Security

Beyond the DOM, browsers give JavaScript a large toolbox: talking to servers (`fetch`), remembering things (Web Storage, cookies, IndexedDB), scheduling visual updates (`requestAnimationFrame`), managing URLs and history, running code on other threads (separate lines of execution that run at the same time — Web Workers), and watching elements (observers). The same power makes web pages a target, so the second half of this chapter covers the security model — the same-origin policy and CORS — and the three attacks every web developer must be able to prevent: XSS (cross-site scripting), CSRF (cross-site request forgery), and the defence-in-depth (an extra protective layer in case another one fails) header CSP (Content Security Policy).

**What you'll learn**

1. `fetch`: requests, responses, JSON, headers, HTTP vs network errors, timeouts.
2. The same-origin policy and CORS, including preflight requests (demonstrated in Chrome).
3. `localStorage` and `sessionStorage`.
4. Cookies and IndexedDB, and how to choose a storage mechanism.
5. Timers vs `requestAnimationFrame` for smooth animation.
6. `URL` and `URLSearchParams`.
7. The History API and client-side routing.
8. Web Workers for CPU-heavy work.
9. `IntersectionObserver` and `ResizeObserver`.
10. XSS and how to sanitize; CSRF and how to block it; Content Security Policy.

**Before you start.** Promises and `async`/`await` (Chapter 12), and DOM/events (Chapter 13). JSON is covered in Chapter 5; error classes in Chapter 11.

**How the code runs.** Programs in `code/` run in Node 24. `fetch` demos start a real **`node:http` server inside the same script** on a free local port (a numbered "door" on a machine that one network program listens on), so nothing touches the internet. Storage, URL and History demos use jsdom (`tools_dom.mjs`, a JavaScript implementation of the browser DOM for Node), which implements `localStorage`, `document.cookie` and `history`. Features that need a real browser — CORS enforcement, CSP, Web Workers, observers, IndexedDB — are small pages served by `code/10_browser_demo_server.mjs` on `localhost:4101` (`localhost` = your own computer; this one is the "site") and `localhost:4102` (a second origin, the "API"), run in headless Chrome (Chrome with no visible window, driven by a script) and screenshotted.

---

## 14.1 `fetch` and HTTP

### What it is

`fetch(url, options?)` sends an HTTP (HyperText Transfer Protocol — the request/response language browsers and web servers speak; see System Design Fundamentals) request and returns a promise for a **`Response`**. It's built into browsers and, since Node 18, into Node (same API). It replaced the callback-based `XMLHttpRequest` (XHR).

HTTP in one paragraph: a **request** has a **method** (`GET` read, `POST` create, `PUT`/`PATCH` update, `DELETE` remove), a URL, **headers** (metadata — data about the request — like `Content-Type`, the body's format, and `Authorization`, the caller's credentials) and optionally a **body**. The **response** has a **status code** (2xx success, 3xx redirect — "look elsewhere", 4xx client error such as 404 Not Found or 401 Unauthorized, 5xx server error), headers and a body.

### How it works

1. `fetch` resolves as soon as the **headers** arrive — with a `Response`, whatever the status code.
2. You read the body with one of `res.json()`, `res.text()`, `res.blob()` (a Blob is raw binary data such as an image), `res.arrayBuffer()`, `res.formData()` — each returns another promise, and a body can be read **once**.
3. `fetch` **rejects only on network failure** (DNS — the system that turns names like `example.com` into IP addresses — failing, refused connection, CORS block, abort). A 404 or 500 is a successful HTTP exchange: check `res.ok` (status 200–299).

`code/01_fetch.mjs` — the server part:

```
const server = http.createServer(async (req, res) => {
  const url = new URL(req.url, 'http://localhost');
  const send = (status, body, headers = {}) => {
    res.writeHead(status, { 'Content-Type': 'application/json', ...headers });
    res.end(JSON.stringify(body));
  };
  if (req.method === 'GET' && url.pathname === '/api/users') return send(200, users, { 'X-Total-
Count': String(users.length) });
  if (req.method === 'POST' && url.pathname === '/api/users') { /* read body, push, send(201, user, {
Location }) */ }
  if (url.pathname === '/api/slow') return setTimeout(() => send(200, { ok: true }), 2000);
  if (url.pathname === '/api/crash') return send(500, { error: 'database unavailable' });
  if (url.pathname === '/page') { res.writeHead(200, { 'Content-Type': 'text/html' }); return
res.end('<!doctype html><h1>Not JSON</h1>'); }
  send(404, { error: `no route ${req.method} ${url.pathname}` });
});
await new Promise((r) => server.listen(0, '127.0.0.1', r));   // port 0 = any free port
const BASE = `http://127.0.0.1:${server.address().port}`;
```

The client part:

```
// 1) GET + JSON
const res = await fetch(`${BASE}/api/users`);
console.log('1) status:', res.status, res.statusText, '| ok:', res.ok, '| type:',
res.headers.get('content-type'), '| total:', res.headers.get('x-total-count'));
console.log('   body  :', await res.json());

// 2) POST JSON
const created = await fetch(`${BASE}/api/users`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ name: 'Grace' }),
});
console.log('2) POST ->', created.status, created.headers.get('location'), await created.json());

// 3) HTTP errors do NOT reject: check res.ok yourself
const notFound = await fetch(`${BASE}/api/nope`);
console.log('3) 404 resolved, ok =', notFound.ok, '| body:', (await notFound.json()).error);

// 4) A reusable helper that turns HTTP errors into exceptions
class HttpError extends Error {
  constructor(res, body) { super(`HTTP ${res.status} for ${res.url}`); this.name = 'HttpError';
this.status = res.status; this.body = body; }
}
async function getJSON(path, init) {
  const r = await fetch(`${BASE}${path}`, init);
  const isJson = r.headers.get('content-type')?.includes('application/json');
  const body = isJson ? await r.json() : await r.text();
  if (!r.ok) throw new HttpError(r, body);
  if (!isJson) throw new TypeError(`expected JSON, got ${r.headers.get('content-type')}`);
  return body;
}

// 5) Network failure DOES reject (TypeError) - a port where nothing listens
await fetch(`http://127.0.0.1:${deadPort}/`).catch((e) => console.log('5) network error:', e.name, '-
', e.message, '| cause:', e.cause?.code));

// 6) Timeout / cancel with AbortSignal
await fetch(`${BASE}/api/slow`, { signal: AbortSignal.timeout(100) })
  .catch((e) => console.log('6) aborted:', e.name, /* ... */));

// 7) A body can be read only once; clone() first if you need it twice
const r7 = await fetch(`${BASE}/api/users`);
const copy = r7.clone();
await r7.json();
await r7.text().catch((e) => console.log('7) second read:', e.name, '-', e.message));

// 8) Query strings, other methods, Request/Headers objects
const q = new URL('/api/nope', BASE);
q.searchParams.set('search', 'Ada Lovelace & co');
const req = new Request(`${BASE}/api/users`, { method: 'DELETE', headers: new Headers({ Authorization:
'Bearer demo-token' }) });
```

Output:

```
1) status: 200 OK | ok: true | type: application/json | total: 1
   body  : [ { id: 1, name: 'Ada' } ]
2) POST -> 201 /api/users/2 { id: 2, name: 'Grace' }
3) 404 resolved, ok = false | body: no route GET /api/nope
4) /api/users -> 2 users
4) /api/crash -> HttpError | 500 database unavailable
4) /page -> TypeError |  expected JSON, got text/html
5) network error: TypeError - fetch failed | cause: ECONNREFUSED
6) aborted: TimeoutError after ~100 ms
7) second read: TypeError - Body is unusable: Body has already been read
   clone still readable: 2 users
8) url: /api/nope?search=Ada+Lovelace+%26+co
   Request: DELETE Bearer demo-token -> 404
```

(Browsers word the network error as `TypeError: Failed to fetch` (Chrome) or `NetworkError when attempting to fetch resource` (Firefox); Node says `fetch failed` and puts the system error in `cause`.)

### Useful `fetch` options

| Option | Values | Purpose |
|---|---|---|
| `method` | `'GET'` (default), `'POST'`, … | HTTP method |
| `headers` | object or `Headers` | e.g. `Content-Type`, `Authorization`, `Accept` |
| `body` | string, `FormData`, `URLSearchParams`, `Blob`, `ArrayBuffer`, stream | Not allowed with GET/HEAD. `FormData` sets a multipart (several parts, e.g. fields plus files) `Content-Type` for you — don't set it yourself |
| `signal` | `AbortSignal` | Cancel / timeout (§12.11) |
| `credentials` | `'same-origin'` (default), `'include'`, `'omit'` | Whether cookies are sent (§14.2) |
| `mode` | `'cors'` (default), `'same-origin'`, `'no-cors'` | CORS behaviour |
| `cache` | `'default'`, `'no-store'`, `'reload'`, … | HTTP cache use (the browser's saved copies of earlier responses) |
| `keepalive` | `true` | Let a small request outlive the page (analytics — usage tracking — sent as the page closes) |

### Common mistakes

- Not checking `res.ok` → your code parses an error page, or treats a 500 as data.
- Calling `res.json()` on a non-JSON response → `SyntaxError: Unexpected token '<'` (an HTML error page) — Chapter 11 §11.10.
- Forgetting `Content-Type: application/json` when sending JSON — many servers won't parse the body.
- Building query strings by concatenation (`'?q=' + text`) — `&` or `#` in the text breaks the URL. Use `URLSearchParams` (§14.6).
- No timeout: a hung server leaves your UI spinning forever. Always pass a `signal`.
- Putting secrets (API keys — secret strings that identify your app to a paid service) in front-end code (code that runs in the user's browser) — anything shipped to the browser is public. Call your own backend (your server-side code), which holds the key.

### In the interview

> **30-second answer:** "`fetch` returns a promise for a `Response` that resolves when headers arrive, even for 404 or 500 — it only rejects on network errors or aborts, so you check `res.ok`. You read the body once with `json()` or `text()`. For sending JSON you set `Content-Type` and `JSON.stringify` the body. I always wrap it in a helper that throws on non-2xx and use `AbortSignal.timeout` so requests can't hang."

- *Q: Does `fetch` send cookies?* A: For same-origin requests, yes (default `credentials: 'same-origin'`); cross-origin only with `credentials: 'include'` and the right CORS headers.
- *Q: `fetch` vs axios?* A: axios (a popular HTTP-client library) adds conveniences (auto JSON, throw on non-2xx, interceptors — hooks that run on every request or response); `fetch` is built in and a small wrapper covers most needs.

### TL;DR

`fetch` → `Response`; check `res.ok`; read the body once; HTTP errors resolve, network errors reject; send JSON with a header + `JSON.stringify`; always time out.

---

## 14.2 The same-origin policy and CORS

### What it is

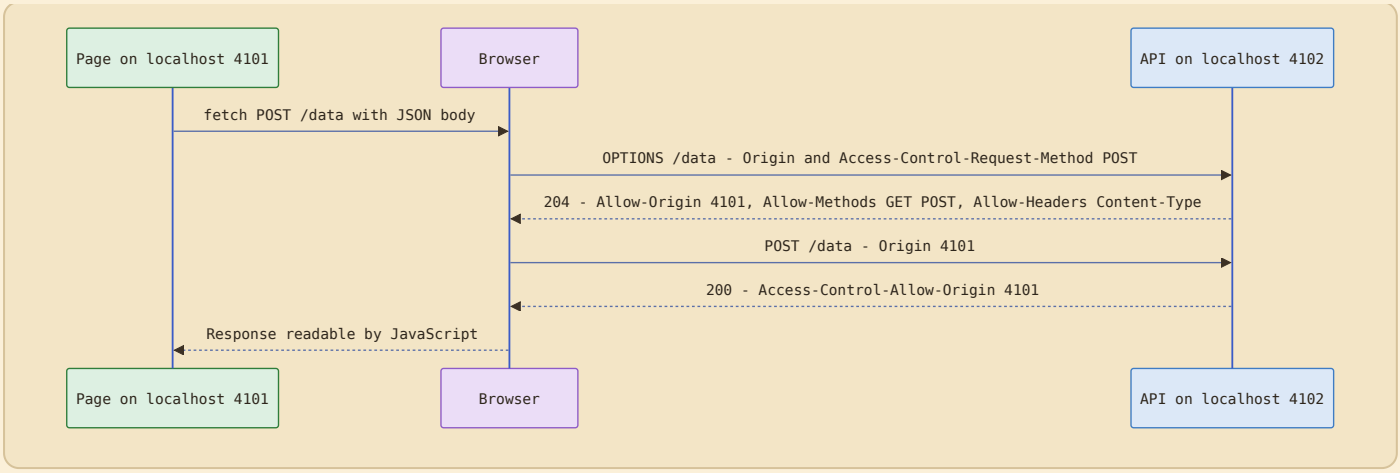
An **origin** is the triple **scheme + host + port** (scheme = `http` or `https`; host = the domain name; port = the number after `:`, 443 by default for HTTPS): `https://shop.example.com:443`. `http://localhost:4101` and `http://localhost:4102` are different origins (different port), as are `http://` vs `https://` and `a.example.com` vs `b.example.com`.

The **same-origin policy (SOP)** is the browser's core security rule: a page may **send** requests anywhere, but its JavaScript may **read responses** only from its own origin. Without it, any website you visit could `fetch('https://your-bank.example/account')` — your browser would attach your bank cookies — and read your balance.

**CORS (Cross-Origin Resource Sharing)** is the controlled exception: a server can declare, with response headers, which other origins may read its responses.

### How it works

1. The page calls `fetch('http://localhost:4102/public')` from origin `http://localhost:4101`.
2. The browser adds an `Origin: http://localhost:4101` request header.
3. **Simple requests** (GET/HEAD/POST with only "safelisted" headers, and `Content-Type` limited to `text/plain`, `multipart/form-data` or `application/x-www-form-urlencoded`) are sent directly.
4. Anything else — `PUT`/`DELETE`, a JSON `Content-Type`, an `Authorization` header — first triggers a **preflight**: an automatic `OPTIONS` request asking "may I?" with `Access-Control-Request-Method`/`-Headers`. The real request is sent only if the answer allows it.
5. When the response arrives, the browser checks `Access-Control-Allow-Origin`. If it matches the page's origin (or is `*`), JavaScript gets the response; otherwise `fetch` rejects with a `TypeError` and the details go only to the DevTools console.




The CORS part of the API server in `code/10_browser_demo_server.mjs`:

```
const allow = { 'Access-Control-Allow-Origin': SITE, Vary: 'Origin' };   // SITE =
'http://localhost:4101'
if (req.method === 'OPTIONS') {                          // preflight: allow POST with a JSON body,
but not PUT
  return res.writeHead(204, { ...allow, 'Access-Control-Allow-Methods': 'GET, POST',
    'Access-Control-Allow-Headers': 'Content-Type', 'Access-Control-Max-Age': '600' }).end();
}
if (path === '/public') return json(200, { msg: 'hello from another origin' }, allow);
if (path === '/private') return json(200, { secret: 'no CORS header on this one' });   // browser will
hide it
if (path === '/data') return json(200, { saved: true, method: req.method }, allow);
```

Run without arguments, the script inspects those headers with Node's `fetch` — which does **not** enforce CORS (only browsers do):

Output:

```
preflight -> 204 {
  'access-control-allow-headers': 'Content-Type',
  'access-control-allow-methods': 'GET, POST',
  'access-control-allow-origin': 'http://localhost:4101',
  'access-control-max-age': '600'
}
/public  ACAO: http://localhost:4101
/private ACAO: null | Node still reads it: no CORS header on this one
csp.html CSP : default-src 'self'; script-src 'self'; img-src 'self'; style-src 'self' 'unsafe-inline'
```

In Chrome, the page `code/cors.html` (served from `:4101`) makes five requests to `:4102`, then asks the API what it received:

**Page origin** `http://localhost:4101` **calling API origin** `http://localhost:4102`

| Request from the page | What JavaScript got |
|---|---|
| GET /public (server sends Access-Control-Allow-Origin) | **200 OK** `{"msg":"hello from another origin"}` |
| GET /private (no CORS header) | **TypeError: Failed to fetch** (details only in the DevTools console) |
| POST /data with JSON body (needs preflight) | **200 OK** `{"saved":true,"method":"POST"}` |
| PUT /data (preflight does not allow PUT) | **TypeError: Failed to fetch** (details only in the DevTools console) |
| GET /private with mode: "no-cors" | **opaque response**: status 0, body hidden |

**What the API server actually received**

| # | Request |
|---|---|
| 1 | `GET /public` |
| 2 | `GET /private` |
| 3 | `OPTIONS /data (preflight for POST)` |
| 4 | `POST /data` |
| 5 | `OPTIONS /data (preflight for PUT)` |
| 6 | `GET /private` |

*Figure — CORS as enforced by Chrome, and what actually reached the server.*

What the screenshot proves:

- `/private` **reached the server** (log line 2) — CORS doesn't stop requests, it stops the page **reading** the response. That's why CORS is not a defence against CSRF (§14.11).
- The JSON `POST` caused an automatic `OPTIONS` preflight first (lines 3–4).
- For `PUT`, the preflight said no, so the real `PUT` was **never sent** (line 5 has no follow-up).
- `mode: 'no-cors'` gives an **opaque** response: status 0, unreadable body — useful only for fire-and-forget requests.

### CORS headers

| Header (response) | Meaning |
|---|---|
| `Access-Control-Allow-Origin` | `*` or one exact origin (not a list — echo back an allow-listed (approved) `Origin` and add `Vary: Origin`, which tells caches the answer differs per origin) |
| `Access-Control-Allow-Methods` / `-Headers` | Preflight answer: allowed methods and request headers |
| `Access-Control-Allow-Credentials: true` | Allow cookies with `credentials: 'include'`; then `Allow-Origin` **can't** be `*` |
| `Access-Control-Max-Age` | Seconds the browser may cache the preflight answer |
| `Access-Control-Expose-Headers` | Non-standard response headers JS may read (e.g. `X-Total-Count`) |

## Common mistakes

- "Fixing" CORS in the front end — impossible; the **server** must send the headers (or proxy the API through your own origin — a proxy is a server that forwards requests on your behalf — e.g. the

dev-server proxy in Vite, a popular front-end build tool).

- `Access-Control-Allow-Origin: *` plus credentials — browsers reject it.
- Reflecting any `Origin` back with credentials allowed — effectively disables the SOP for your API. Use an allow-list.
- Thinking a CORS error means the server is down — it usually means the server answered without the right headers.

### In the interview

> **30-second answer:** "The same-origin policy lets a page send requests anywhere but only read responses from its own scheme-host-port. CORS lets a server opt in with `Access-Control-Allow-Origin`. Non-simple requests — JSON bodies, custom headers, PUT/DELETE — first get an OPTIONS preflight. CORS is enforced by the browser, not the server, and it protects reading, not sending."

- *Q: Why does Postman/curl (tools for sending HTTP requests by hand) work but the browser fails?* A: Only browsers enforce CORS.
- *Q: Is CORS a security feature for the server?* A: No, it's a relaxation of the browser's SOP; servers still need authentication (checking who the caller is) and CSRF protection.

### TL;DR

Origin = scheme + host + port. SOP blocks reading cross-origin responses; CORS headers relax it per origin. Non-simple requests get an OPTIONS preflight. Browsers enforce it; the server configures it.

---

## 14.3 Web Storage: `localStorage` and `sessionStorage`

### What it is

Two simple **key–value stores** (each value is saved and looked up under a name, its key) of **strings**, per origin, with a synchronous API (calls finish before returning, no promises):

- **`localStorage`** — persists until cleared (by your code or the user); shared by all tabs of the origin.
- **`sessionStorage`** — per **tab** (and origin); cleared when the tab closes; survives reloads.

API: `setItem(key, value)`, `getItem(key)` (→ `null` if missing), `removeItem(key)`, `clear()`, `key(i)`, `length`. Changes made in one tab fire a **`storage`** **event** in the **other** tabs of the same origin (not the one that wrote) — handy for syncing logout or theme across tabs.

`code/03_storage.mjs` — key parts:

```
localStorage.setItem('theme', 'dark');
localStorage.count = 5;                              // property style works but is discouraged
console.log('   numbers become strings:', typeof localStorage.getItem('count'),
JSON.stringify(localStorage.getItem('count')));
localStorage.setItem('user', { name: 'Ada' });       // WRONG: stores "[object Object]"

const store = {
  get(key, fallback = null) {
    try { const raw = localStorage.getItem(key); return raw === null ? fallback : JSON.parse(raw); }
    catch { return fallback; }                           // corrupt value or storage disabled
  },
  set(key, value) {
    try { localStorage.setItem(key, JSON.stringify(value)); return true; }
    catch (e) { console.log('   set failed:', e.name); return false; }  // QuotaExceededError, private
mode...
  },
};
store.set('cart', { items: [{ sku: 'A-1', qty: 2 }], updated: '2026-09-28' });
// ... enumerate, removeItem
store.set('huge', 'x'.repeat(6_000_000));            // over the ~5 MB quota
sessionStorage.setItem('wizardStep', '2');
```

Output (lines 1–6):

```
1) getItem      : dark | missing -> null
   numbers become strings: string "5"
   object stored raw     : [object Object]
2) JSON round trip: { sku: 'A-1', qty: 2 } | corrupt -> fallback
3) length: 5 | keys: [ 'theme', 'count', 'user', 'cart', 'bad' ]
   after removeItem: 4
   set failed: QuotaExceededError
5) sessionStorage: 2 | separate from localStorage: null
6) after clear(): 0
```

## Trade-offs

| ✓ Good for | ✗ Not for |
|---|---|
| UI preferences (theme, collapsed panels), drafts, small caches | **Secrets and auth tokens** (strings proving a user is logged in) — any script on the page (including an XSS payload — the attacker's injected code — or a compromised npm package, i.e. a hacked library) can read it |
| Data that must survive reloads, a few KB–MB | Large data (quota ~5 MB per origin; use IndexedDB) |
| | Hot paths (code that runs very often) — it's **synchronous** and blocks the main thread on disk access |
| | Data for the server — it's never sent automatically (unlike cookies) |

## Common mistakes

- Storing objects without `JSON.stringify` → `"[object Object]"`.
- Not wrapping in `try`/`catch`: quota errors, and storage can be disabled (some privacy modes, sandboxed iframes — embedded pages with restricted permissions) so even *reading* can throw.
- Expecting `sessionStorage` to be shared across tabs — it isn't (a duplicated tab gets a copy).

**TL;DR**

String key–value storage per origin; `localStorage` persists, `sessionStorage` is per tab. JSON + try/catch; never store secrets.

---

## 14.4 Cookies and IndexedDB — choosing storage

### Cookies

A **cookie** is a small (≈4 KB) name=value pair the browser stores per site and **sends automatically with every matching HTTP request** in the `Cookie` header. Servers set them with `Set-Cookie`; JavaScript can read/write non-HttpOnly cookies (ones without the `HttpOnly` flag, which hides a cookie from scripts) through the awkward `document.cookie` string.

```
document.cookie = 'lang=en; Path=/; Max-Age=86400; SameSite=Lax';
document.cookie = 'consent=yes; Path=/';
document.cookie = 'session=secret; HttpOnly';        // IGNORED: HttpOnly cookies can only be set by
the server
console.log('7) document.cookie:', JSON.stringify(document.cookie));
const cookies = Object.fromEntries(document.cookie.split('; ').map((c) =>
c.split('=').map(decodeURIComponent)));
document.cookie = 'lang=; Path=/; Max-Age=0';         // delete = expire it
```

Output (end of `03_storage.mjs`):

```
7) document.cookie: "lang=en; consent=yes"
   parsed: { lang: 'en', consent: 'yes' }
   after delete: "consent=yes"
```

Assigning to `document.cookie` adds or updates **one** cookie; reading returns all visible ones as one string, without their attributes.

| Attribute | Meaning |
|---|---|
| `Expires=<date>` / `Max-Age=<seconds>` | Lifetime; without either it's a **session cookie** (deleted when the browser session ends) |
| `Domain` / `Path` | Which hosts/paths receive it (omit `Domain` = this host only, the safest) |
| `Secure` | Only sent over HTTPS (encrypted HTTP) (browsers treat `localhost` as secure for development) |
| `HttpOnly` | Invisible to JavaScript — protects session cookies from theft by XSS |
| `SameSite=Strict` / `Lax` / `None` | Whether it's sent on **cross-site** requests (§14.11). `None` requires `Secure`. Chrome and Edge treat a missing attribute as `Lax`; set it explicitly |
| `__Host-` name prefix | Browser enforces `Secure`, `Path=/`, no `Domain` — hardest to tamper with |

For session IDs (random strings the server uses to recognise a logged-in user) the standard recipe is: `Set-Cookie: __Host-sid=…; HttpOnly; Secure; SameSite=Lax; Path=/`. (A newer asynchronous `cookieStore` API exists; check browser support before using it.)

### IndexedDB

**IndexedDB** is the browser's real database: asynchronous, transactional (a group of changes either all happen or none do), storing **structured-cloneable** values (ones the browser's built-in deep-copy algorithm can copy: objects, arrays, `Date`, `Map`, `Blob`, typed arrays) in **object stores** (roughly,

tables) with **keys** and **indexes** (extra lookup structures for searching fast by a field), with quotas typically in the hundreds of MB to GB range. Its raw API is event-based and verbose, so most apps use a thin promise wrapper (like the small `req2p` below, or the `idb` library) or a higher-level library (Dexie).

The page `code/idb.html` — key parts:

```
const req2p = (req) => new Promise((resolve, reject) => { req.onsuccess = () => resolve(req.result);
req.onerror = () => reject(req.error); });

const open = indexedDB.open('notes-demo', 1);
open.onupgradeneeded = () => {                              // runs when the version number increases
  const store = open.result.createObjectStore('notes', { keyPath: 'id', autoIncrement: true });
  store.createIndex('byTag', 'tag');
};
const db = await req2p(open);

const tx = db.transaction('notes', 'readwrite');           // all writes commit together, or none
for (const n of notesToAdd) tx.objectStore('notes').add(n);
await new Promise((r) => (tx.oncomplete = r));

const ro = db.transaction('notes').objectStore('notes');   // read-only transaction
await req2p(ro.get(2));
await req2p(ro.index('byTag').getAll('home'));             // query by index
await req2p(ro.getAll(IDBKeyRange.bound(2, 3)));           // key range
```

**IndexedDB: a notes database in the browser**

```
upgradeneeded: created store "notes" + index "byTag"
added 3 notes (Dates are stored as real Date objects — structured clone)
count      : 3
get(2)     : Ship chapter 14 | created is a Date: true
tag="home" : Buy milk, Fix bike
ids 2..3   : 2:Ship chapter 14, 3:Fix bike
storage.estimate(): quota is about 11 GB for this origin (varies by browser and disk)
```

*Figure — IndexedDB in headless Chrome. The quota comes from `navigator.storage.estimate()` and depends on the browser and free disk space.*

### Choosing

| | Cookies | localStorage | sessionStorage | IndexedDB |
|---|---|---|---|---|
| Capacity | ~4 KB each | ~5 MB/origin | ~5 MB/origin | Large (share of disk) |
| API | String; sync | Sync | Sync | Async, transactions |
| Data types | Strings | Strings | Strings | Structured clone (objects, Dates, Blobs…) |
| Sent to server | **Automatically** | No | No | No |
| Lifetime | Expiry / session | Until cleared | Tab | Until cleared (may be evicted — deleted by the browser — under storage pressure unless `navigator.storage.persist()` is granted) |
| Readable by JS | Unless `HttpOnly` | Yes | Yes | Yes |
| Typical use | Session ID, CSRF, consent | Preferences | Wizard progress | Offline data, caches, large datasets |

A fifth option, the **Cache API**, stores whole HTTP responses; service workers (background scripts that sit between a page and the network) use it to make apps work offline (§14.8).

#### TL;DR

Cookies = tiny, auto-sent, the right place for `HttpOnly` session IDs. Web Storage = small, sync, strings. IndexedDB = async database for big or structured data.

---

## 14.5 Timers and `requestAnimationFrame`

#### What it is

`requestAnimationFrame(callback)` (**rAF**) asks the browser to run `callback(timestamp)` **just before the next repaint** — once per frame, matching the display's refresh rate (how many times per second the screen redraws; 60 Hz → every ~16.7 ms, 120 Hz → ~8.3 ms). It's the right scheduler for anything visual. Timers (`setTimeout`/`setInterval`, Chapter 12 §12.5) are for everything else.

| | setTimeout / setInterval | requestAnimationFrame |
|---|---|---|
| When | After a delay, as a task | Right before paint, in the render step of the event loop |
| Synced to display | No (can run twice per frame or miss frames → jank, visible stutter) | Yes |
| Background tabs | Throttled (≥1 s typically) | **Paused** — saves battery |
| Callback argument | Your args | High-resolution frame timestamp |
| Cancel | `clearTimeout` | `cancelAnimationFrame(id)` |

`code/06_raf.mjs` (jsdom simulates 60 fps):

```
function animate(el, to, duration) {
  return new Promise((resolve) => {
    let start;
    const frames = [];
    function step(now) {                        // now = frame timestamp (ms), same clock as
performance.now()
      start ??= now;
      const t = Math.min((now - start) / duration, 1);
      const eased = 1 - (1 - t) ** 3;           // ease-out cubic
      el.style.width = `${Math.round(eased * to)}px`;
      frames.push(el.style.width);
      if (t < 1) requestAnimationFrame(step);
      else resolve(frames);
    }
    requestAnimationFrame(step);
  });
}
const frames = await animate(bar, 300, 200);

// Coalesce many events into one update per frame (e.g. scroll or pointermove)
let scheduled = false;
function onPointerMove() {
  events++;
  if (scheduled) return;
  scheduled = true;
  requestAnimationFrame(() => { scheduled = false; updates++; /* read layout, then write styles */ });
}
```

Output:

```
1) about 12 frames in 200 ms at 60 fps: true
   first widths: 0px, 69px, 130px, 178px ... last: 300px
2) cancelled a frame callback
3) 50 events -> 1 DOM update(s)
```

The intermediate widths depend on real frame timing, so they differ slightly from run to run; the first (0px) and last (300px) values do not.

### Rules

1. **Base progress on time, not frame count** — frame rates vary (60/120/144 Hz, dropped frames). Compute `t = elapsed / duration` from the timestamp.
2. rAF runs **once**; call it again inside the callback to continue.
3. Use it to **coalesce** bursts of events (`scroll`, `pointermove`, `resize`) into one DOM update per frame — the third output line.
4. Prefer **CSS transitions/animations** or the Web Animations API (`el.animate()`) for simple effects; they can run on the compositor thread (a separate browser thread that stacks finished page layers onto the screen). Use rAF when JavaScript must compute each frame (games, canvas — an element you draw pixels on from JS — physics, drag).
5. Animate `transform`/`opacity` where possible (Chapter 13 §13.11).

> **TL;DR**

rAF = "run before the next paint", once per frame, paused in background tabs. Time-based progress, re-request each frame, coalesce events through it.

---

## 14.6 `URL` and `URLSearchParams`

### What it is

`new URL(input, base?)` parses a URL into parts; `url.searchParams` is a `URLSearchParams` for the query string (the `?key=value&…` part). Both handle **percent-encoding** (writing unsafe characters as `%` plus a hex code, e.g. a space as `%20`) correctly, so you never glue URLs together with `+`.

`code/04_url.mjs` — key parts:

```
const u = new URL('https://shop.example.com:8443/search/results?q=red+shoes&size=9&size=10#reviews');
console.log('1) parts:', { protocol: u.protocol, host: u.host, hostname: u.hostname, port: u.port,
pathname: u.pathname, search: u.search, hash: u.hash, origin: u.origin });
const p = u.searchParams;
p.set('page', '2'); p.append('size', '11'); p.delete('q'); p.sort();
console.log('3)', new URL('../img/logo.png', 'https://example.com/docs/guide/').href);
const search = new URLSearchParams({ q: 'Tom & Jerry', tag: 'c++', emoji: '✓' });

function safeHttpUrl(input) {
  try {
    const url = new URL(input);
    return ['http:', 'https:'].includes(url.protocol) ? url.href : null;
  } catch { return null; }
}
```

Output:

```
1) parts: {
  protocol: 'https:',
  host: 'shop.example.com:8443',
  hostname: 'shop.example.com',
  port: '8443',
  pathname: '/search/results',
  search: '?q=red+shoes&size=9&size=10',
  hash: '#reviews',
  origin: 'https://shop.example.com:8443'
}
2) q = red shoes | size = 9 | getAll = [ '9', '10' ] | has page: false
   now: ?page=2&size=9&size=10&size=11 | entries: [
  [ 'page', '2' ],
  [ 'size', '9' ],
  [ 'size', '10' ],
  [ 'size', '11' ]
]
3) https://example.com/docs/img/logo.png
   https://example.com/api?x=1
4) q=Tom+%26+Jerry&tag=c%2B%2B&emoji=%E2%9C%93
   manual concat would break: q=Tom & Jerry <- the & starts a new parameter
   encodeURIComponent: Tom%20%26%20Jerry%2F%3F | encodeURI keeps / and ?: /a%20b/?x=1&y=2
5) https://ada.dev          -> https://ada.dev/
5) javascript:alert(1)      -> null
5) not a url                -> null
5) data:text/html,<b>x</b>  -> null
   URL.canParse: true false
6) { a: '1', b: 'two' } | k=v1&k=v2
```

### Notes

- `new URL('relative')` **without a base throws** `TypeError: Invalid URL`; `URL.canParse()` checks without throwing.
- `URLSearchParams` encodes spaces as `+` (form encoding); `encodeURIComponent` uses `%20`. Both decode correctly on servers.
- `get` returns the first value; `getAll` all of them — repeated keys are normal (`?size=9&size=10`).
- In the browser, `location` is URL-like (`location.pathname`, `location.search`), and `new URL(location.href)` gives you a mutable (changeable) copy.
- **Validate user-supplied URLs** by parsing and allow-listing the protocol (item 5). Blocking `javascript:` URLs prevents a classic XSS (§14.10).

#### TL;DR

Parse with `new URL`, build queries with `URLSearchParams`, resolve relative URLs with a base, and allow-list protocols for user links.

---

## 14.7 The History API

#### What it is

The **History API** lets a page change the URL in the address bar and add entries to the back/forward list **without loading a new page** — the foundation of **single-page application (SPA)** routing — an SPA loads one HTML page and swaps views with JavaScript; *routing* maps each URL to a view (React Router, Vue Router).

| API | Does |
|---|---|
| `history.pushState(state, '', url)` | Add a new entry and change the URL (same origin only). No request, no `popstate` |
| `history.replaceState(state, '', url)` | Change the current entry (no new entry) — e.g. update query params for filters |
| `history.state` | The state object of the current entry (structured-cloned) |
| `history.back()` / `forward()` / `go(n)` | Navigate; async |
| `popstate` event on `window` | Fires when the user (or code) moves back/forward to an entry — **re-render from** `location` |

`code/05_history.mjs` — a 30-line router:

```
const routes = [
  [/^\/$/, () => 'Home page'],
  [/^\/products$/, () => 'All products'],
  [/^\/products\/(\d+)$/, (id) => `Product #${id}`],
];
function render() { /* match location.pathname against routes, write the view with textContent */ }

// Intercept same-origin link clicks (delegation) and navigate without reloading
document.addEventListener('click', (e) => {
  const a = e.target.closest('a');
  if (!a || a.origin !== location.origin || e.ctrlKey || e.metaKey) return;   // let the browser handle it
  e.preventDefault();
  history.pushState({ from: location.pathname }, '', a.href);                  // new history entry, no request
  render();
});
// Back/forward buttons fire popstate: re-render from the URL
window.addEventListener('popstate', () => { render(); show('popstate (back/fwd)'); });

render(); show('initial');
document.querySelector('a[href="/products"]').click(); show('click Products');
document.querySelector('a[href="/products/42"]').click(); show('click Lamp');
history.replaceState({ tab: 'reviews' }, '', '/products/42?tab=reviews'); show('replaceState (no new entry)');
history.back();
```

Output:

```
initial                     /              | Home page | state: null | length: 1
click Products              /products      | All products | state: {"from":"/"} | length: 2
click Lamp                  /products/42   | Product #42 | state: {"from":"/products"} | length: 3
replaceState (no new entry) /products/42   | Product #42 | state: {"tab":"reviews"} | length: 3
popstate (back/fwd)         /products      | All products | state: {"from":"/"} | length: 3
external link: same origin? false -> not intercepted
```

### Must-knows for SPA routing

1. **The server must serve your app for every route** (a "fallback" to `index.html`) — otherwise a reload or shared link to `/products/42` gets a 404.
2. Respect modifier keys (Ctrl, Cmd, Shift — e.g. Ctrl+click opens a new tab), `target="_blank"` and external links (let the browser do its normal thing).
3. Manage **focus and scroll** after navigation (move focus to the new heading; restore scroll on back) — screen-reader users otherwise don't notice the page changed.
4. Put UI state that should be shareable (filters, tabs, search) in the URL (`replaceState` + `URLSearchParams`), not only in memory.
5. Hash routing (`#/products`) + the `hashchange` event is the older approach that needs no server config.

A newer **Navigation API** (`navigation.navigate()`, a single `navigate` event for all navigations) simplifies routers; check browser support before relying on it.

### TL;DR

`pushState`/`replaceState` change the URL without loading; `popstate` fires on back/forward — re-render from `location`. Configure the server fallback; handle focus.

---

## 14.8 Web Workers

### What it is

A **Web Worker** runs a script on a **separate background thread** with its own event loop and global scope (`self` — its top-level object, like `window` on a page). It has no access to the DOM, `window` or `document`, but it has `fetch`, timers, IndexedDB, `crypto` (built-in cryptography: hashing, random numbers, encryption) and more. The page and the worker communicate only by **messages**:

```
// main page
const w = new Worker('prime_worker.js');
w.onmessage = (e) => show(e.data.count);
w.postMessage(5_000_000);
```

```
// prime_worker.js
self.onmessage = (e) => {
  const count = countPrimes(e.data);
  self.postMessage({ upTo: e.data, count });
};
```

Messages are copied with the **structured clone algorithm** (plain objects, arrays, `Date`, `Map`, `Set`, `Blob`, typed arrays… but **not** functions, DOM nodes or class prototypes). Big binary buffers (blocks of raw bytes, `ArrayBuffer`) can be **transferred** instead of copied (`postMessage(buf, [buf])` — the sender loses access, zero-copy: ownership moves, no bytes are duplicated).

### Why it matters — measured

`code/worker.html` counts primes up to 5 million twice: once on the main thread, once in `code/prime_worker.js`. A 10 ms `setInterval` tick counter measures whether the page could do anything else meanwhile:

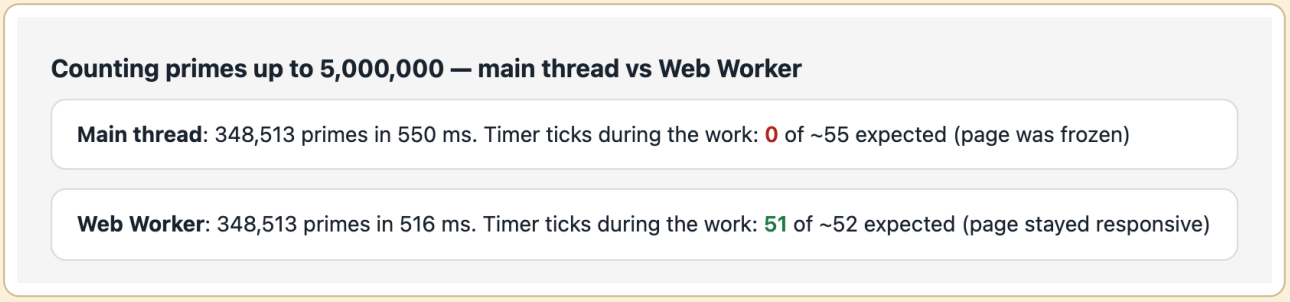



*Figure — same work, same time; only the worker leaves the main thread free (headless Chrome).*

The worker doesn't make the computation faster — it moves it off the thread that handles clicks, scrolling and painting. Node has the same model in `worker_threads`; `code/07_workers_node.mjs` shows the message passing and structured-clone rules:

```
const worker = new Worker(workerSource, { eval: true });
const sample = { when: new Date(0), tags: new Map([['a', 1]]) };   // Date and Map survive structured cloning
const ticker = setInterval(() => ticks.push('tick'), 5);            // the main thread stays responsive
const result = await new Promise((resolve, reject) => {
  worker.once('message', resolve);
  worker.once('error', reject);
  worker.postMessage({ upTo: 3_000_000, sample });
});
try { worker.postMessage({ fn: () => 1 }); } catch (e) { console.log('clone error   :', e.name); }
```

Output:

```
worker result : { upTo: 3000000, count: 216816, sampleIsCopy: true, mapSize: 1 }
main thread kept running while the worker computed: true
clone error   : DataCloneError
structuredClone keeps Date/Map: true
```

**Kinds of workers**

| Kind | Purpose |
|---|---|
| Dedicated `Worker` | Background computation for one page (this section). `new Worker(url, { type: 'module' })` allows `import` |
| `SharedWorker` | One worker shared by several tabs of the same origin |
| **Service worker** | A network proxy between the page and the server — offline support, caching, push notifications (PWAs — progressive web apps, websites that can be installed and work offline like native apps). Event-driven, not for computation |
| Worklets | Tiny specialized scripts (audio, CSS paint) |

### When not to use it

For I/O (`fetch` is already asynchronous) or small tasks — message copying and worker start-up cost more than they save. Use it for parsing big files, image/audio processing, compression, cryptography, syntax highlighting, search indexes (precomputed lookup tables for fast text search). Libraries like Comlink make the message passing feel like function calls.

### TL;DR

Workers = real parallel threads without DOM access, talking via `postMessage` (structured clone or transfer). Use them to keep CPU-heavy work off the main thread.

---

## 14.9 `IntersectionObserver` and `ResizeObserver`

### What it is

Observers are callbacks the browser calls **when something changes**, instead of you polling in `scroll` or `resize` handlers (which fire constantly and force layout reads).

| Observer | Tells you when… | Typical uses |
|---|---|---|
| `IntersectionObserver` | An element enters/leaves the viewport (the visible part of the page) (or a scroll container), by a given fraction | Lazy loading (loading things only when about to be seen), infinite scroll, "seen" analytics, starting animations |
| `ResizeObserver` | An element's size changes (for any reason, not just window resize) | Responsive components (that adapt their layout to available space), charts that redraw to their container |
| `MutationObserver` | The DOM changes | Chapter 13 §13.10 |

`code/observers.html` — key parts:

```
const io = new IntersectionObserver((entries, obs) => {
  for (const e of entries) {
    if (!e.isIntersecting) continue;
    e.target.classList.add('loaded');
    e.target.textContent = 'item ' + e.target.dataset.i + ' loaded';
    obs.unobserve(e.target);                    // load once
  }
}, { root: scroller, rootMargin: '20px' });     // root: null would mean the viewport
scroller.querySelectorAll('.item').forEach((el) => io.observe(el));

const ro = new ResizeObserver((entries) => {
  for (const e of entries) {
    const { inlineSize, blockSize } = e.contentBoxSize[0];
    log('RO callback: content box ' + Math.round(inlineSize) + ' x ' + Math.round(blockSize) + ' px');
  }
});
ro.observe(box);
// scripted: scrollTop = 300, then 600; then box.style.width = '240px'
```

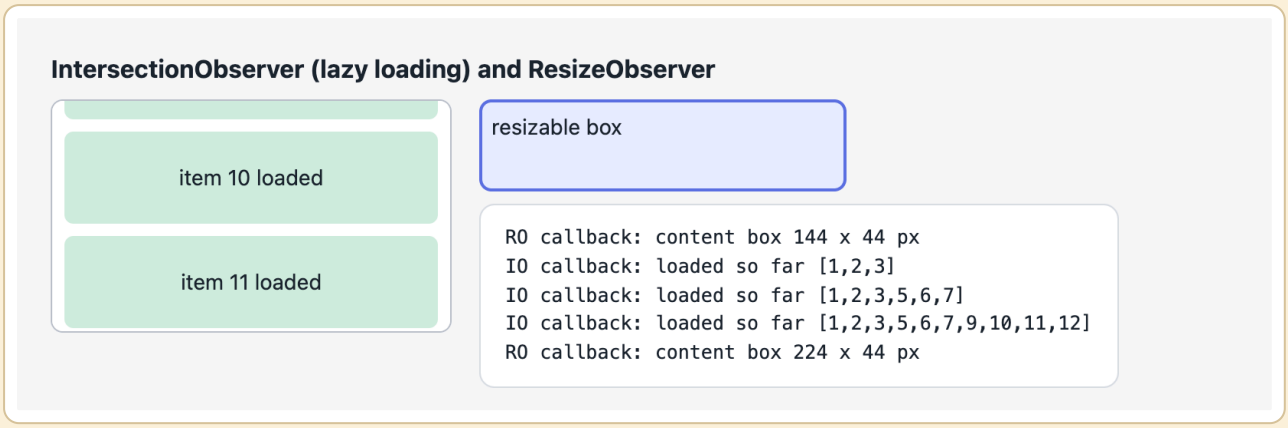



*Figure — observers in headless Chrome. Items 4 and 8 were never loaded: the two programmatic scroll jumps skipped past them, exactly as a fast user scroll would.*

Notes:

- Both observers call back **once initially** for each observed element (the first RO and IO lines), then on changes.
- Options: `root` (container or `null` = viewport), `rootMargin` (grow/shrink the root box, e.g. `'200px'` to start loading early), `threshold` (`0` = any pixel, `1` = fully visible, or an array).
- For images and iframes, prefer the native `loading="lazy"` attribute; use IO for custom cases.
- Always `unobserve` / `disconnect` when done (e.g. in a component's cleanup).
- A `ResizeObserver` callback that changes the observed element's size can loop; the browser stops after one pass and reports "ResizeObserver loop completed with undelivered notifications".

### TL;DR

`IntersectionObserver` for visibility, `ResizeObserver` for element size — no scroll/resize polling, no forced layouts. Disconnect when done.

---

## 14.10 Cross-site scripting (XSS) and sanitizing

### What it is

**Cross-site scripting (XSS)** happens when an attacker gets **their JavaScript to run in your page**, with your page's privileges: reading data shown to the user, calling your APIs as the user, reading non-

HttpOnly cookies and `localStorage`, changing the page (fake login forms). It has appeared in every edition of the OWASP Top 10 (a widely used ranking of the most critical web-application security risks, published by the OWASP non-profit) — as its own category until 2017, and folded into "Injection" since the 2021 edition.

| Type | How the payload arrives |
|---|---|
| **Stored** | Saved on the server (a comment, a profile name) and shown to other users |
| **Reflected** | Echoed from the request (a search term in the URL) into the response |
| **DOM-based** | Client-side code moves untrusted data (URL, `postMessage`, API data) into a dangerous **sink** like `innerHTML` |

### How it works — seen in a real browser

`code/xss.html` renders the same "user comment" twice. The comment contains `<img src="does-not-exist.png" onerror="…">`: the image fails to load, so the browser runs the `onerror` attribute as code.

```
document.getElementById('safe').textContent = comment;   // shown as text
document.getElementById('unsafe').innerHTML = comment;   // parsed: the onerror handler runs
```

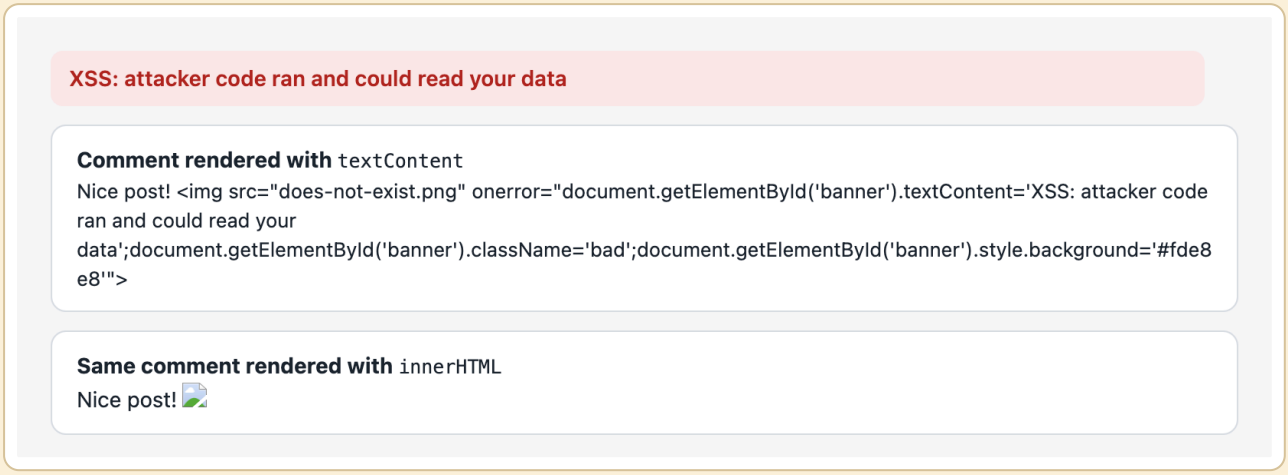



*Figure — one line of `innerHTML` was enough for the attacker's code to take over the page (headless Chrome).*

`code/08_xss.mjs` takes the same payload apart in jsdom:

```
box.innerHTML = comment;
console.log('1) innerHTML created <img> with onerror handler:', img?.getAttribute('onerror'));
box.textContent = comment;
const escapeHtml = (s) => String(s).replace(/[&<>"']/g, (ch) => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;' })[ch]);
box.innerHTML = `<p class="comment">${escapeHtml(comment)}</p>`;
const website = 'javascript:alert(document.cookie)';
a.href = escapeHtml(website);                        // escaping does nothing useful here
```

Output:

```
1) innerHTML created <img> with onerror handler: stealCookies()
   <script> inserted via innerHTML exists but never runs: true (the <img onerror> is what attackers
use instead)
2) textContent -> elements created: 0 | shown literally: Nice post! <img src="x...
3) escaped: &lt;img src=x onerror=&quot;go()&quot;&gt;
   elements inside <p>: 0
4) href after escaping : javascript:alert(document.cookie) <- still runs script when clicked
   href after allow-list: #
5) sinks to avoid with untrusted data:
   - el.innerHTML = s
   - el.outerHTML = s
   - el.insertAdjacentHTML(pos, s)
   - document.write(s)
   - eval(s) / new Function(s)
   - setTimeout('code string')
   - el.setAttribute("on...", s)
   - a.href = s (javascript:)
```

### How to prevent it (in priority order)

1. **Treat all external data as text.** Use `textContent`, `setAttribute` for non-event attributes, and `createElement` — the DOM APIs from Chapter 13 §13.3. Frameworks do this by default: React escapes `{value}` in JSX (React's HTML-like syntax inside JavaScript); the danger is in escape hatches (deliberate ways to bypass a safety feature) (`dangerouslySetInnerHTML`, Vue's `v-html`).
2. **Escape for the context** (escaping = rewriting special characters, like `<` → `<`, so they're shown, not interpreted) when you build HTML strings (server templates): HTML body, attribute, URL and JavaScript contexts each need different encoding (item 4: HTML-escaping doesn't make a `javascript:` URL safe). Use a template engine (a library that fills HTML templates with data) that auto-escapes.
3. **Validate URLs** by parsing and allow-listing `http:`/`https:` (§14.6).
4. **Sanitize** (strip out every dangerous part) **when you must render user HTML** (rich-text comments, Markdown output — Markdown is a lightweight text format that converts to HTML): run it through a well-maintained sanitizer such as **DOMPurify** (`el.innerHTML = DOMPurify.sanitize(html)`), which removes scripts, event-handler attributes and dangerous URLs. The standard **HTML Sanitizer API** (`el.setHTML()`) is still of limited availability across browsers (per MDN, Mozilla's web documentation), so use it only with a fallback. Never write your own regex-based sanitizer.
5. **Defence in depth**: a strict **CSP** (§14.12) so injected code doesn't run even if a bug slips through; `HttpOnly` cookies so sessions can't be stolen; **Trusted Types** (a CSP feature that makes dangerous sinks reject plain strings; check browser support).

### In the interview

**30-second answer:** "XSS is attacker-controlled script running in my origin — stored, reflected or DOM-based. The root cause is untrusted data reaching a sink that parses HTML or code, like `innerHTML`, `eval` or a `javascript:` URL. I prevent it by rendering data as text (`textContent`, framework escaping), context-aware escaping on the server, URL protocol allow-lists, DOMPurify when HTML is truly needed, and a strict CSP plus HttpOnly cookies as backup."

- *Q: Does a `<script>` injected with `innerHTML` run?* A: No — but `<img onerror>`, `<svg onload>` and similar event-handler attributes do, so that's no protection.
- *Q: Is React immune?* A: Mostly for text; not for `dangerouslySetInnerHTML`, `href={userUrl}` with `javascript:` (React warns and newer versions block it), or server-rendered HTML you inject.

### TL;DR

XSS = untrusted data parsed as code. Render as text, escape by context, allow-list URL protocols, sanitize with DOMPurify, add CSP and HttpOnly.

---

## 14.11 Cross-site request forgery (CSRF)

### What it is

**CSRF** tricks a logged-in user's browser into sending a request they didn't intend — for example, a hidden form on `evil.example` that auto-submits `POST https://bank.example/transfer`. Unless the cookie is protected by `SameSite`, the browser attaches the bank's cookies automatically, so the bank sees a valid session. The attacker can't **read** the response (SOP), but for a state-changing request (one that modifies data, like a transfer) they don't need to.

#### How it's prevented

| Defence | How it helps |
|---|---|
| **`SameSite` cookies** (`Lax` or `Strict`) | The browser doesn't send the cookie on cross-site sub-requests and form POSTs. `Lax` still sends it on top-level GET navigations (clicking a link), so **GET must never change state** |
| **CSRF token** (synchronizer token) | The server gives the page a random secret (in the HTML or a JSON response); state-changing requests must echo it in a header or form field. `evil.example` can't read your page, so it can't know the token |
| **Check `Origin` / `Sec-Fetch-Site` headers** | Browsers send the request's origin; reject unexpected ones on state-changing requests |
| **Require a custom header or JSON `Content-Type`** | Forces a CORS preflight for cross-origin requests, which your server won't approve |
| Re-authentication | For critical actions (change password, large transfer) |

Use at least two: `SameSite` + token (or origin check). `code/09_csrf_cookies_headers.mjs` implements session cookies + a token on a local server; since Node's `fetch` has no cookie jar (storage that keeps cookies between requests), the script plays the browser and sends the `Cookie` header by hand:

```
if (req.method === 'POST' && req.url === '/login') {
  const sid = randomBytes(16).toString('hex');
  const csrf = randomBytes(16).toString('hex');
  sessions.set(sid, { user: 'ada', csrf });
  return reply(200, { csrf }, {                          // token goes to the page, cookie to the
browser
    'Set-Cookie': `sid=${sid}; HttpOnly; Secure; SameSite=Lax; Path=/; Max-Age=3600`,
  });
}
if (req.method === 'POST' && req.url === '/transfer') {
  if (!session) return reply(401, { error: 'not logged in' });
  if (!same(req.headers['x-csrf-token'], session.csrf)) return reply(403, { error: 'CSRF token missing
or wrong' });
  return reply(200, { ok: true, by: session.user });
}
// ...
const forged = await fetch(`${BASE}/transfer`, { method: 'POST', headers: { Cookie: cookie } });
const legit = await fetch(`${BASE}/transfer`, { method: 'POST', headers: { Cookie: cookie, 'X-CSRF-
Token': csrf } });
const noCookie = await fetch(`${BASE}/transfer`, { method: 'POST', headers: { 'X-CSRF-Token': csrf }
});
```

Output:

```
1) Set-Cookie: sid=<random>; HttpOnly; Secure; SameSite=Lax; Path=/; Max-Age=3600
   CSP       : default-src 'self'; script-src 'self'; object-src 'none'; base-uri 'self'; frame-
ancestors 'none'
2) forged request  -> 403 CSRF token missing or wrong
3) legit request   -> 200 { ok: true, by: 'ada' }
4) without cookie  -> 401 not logged in
```

Line 2 is a forged request that still carried the cookie (as it would with `SameSite=None`, or on an old browser): the missing token stops it. Line 4 is what `SameSite=Lax` does to a cross-site POST in a modern browser: no cookie, so no session. The token is compared with `crypto.timingSafeEqual` so response timing doesn't leak how many characters matched.

### Common mistakes

- Changing state on `GET` (`/logout`, `/delete?id=5`) — an `<img src>` can trigger it.
- Believing CORS prevents CSRF — CORS controls reading; the forged request is still sent (the CORS screenshot in §14.2 shows `/private` reaching the server).
- Storing the session token in `localStorage` and sending it in an `Authorization` header avoids CSRF but makes the token readable by any XSS — a trade-off, not a free win. `HttpOnly` cookie + `SameSite` + CSRF token is the common recommendation (OWASP CSRF Prevention Cheat Sheet).

### TL;DR

CSRF abuses automatically-sent cookies. Defend with `SameSite` cookies, CSRF tokens (or origin checks), and never change state on GET.

---

## 14.12 Content Security Policy (CSP)

### What it is

**CSP** is an HTTP response header (`Content-Security-Policy`) that tells the browser **which sources of scripts, styles, images, frames etc. are allowed** on this page. Anything else is blocked and

reported. It's **defence in depth**: if an XSS bug lets an attacker inject markup, CSP stops the injected code from running.

Common directives:

| Directive | Controls | Example |
|---|---|---|
| `default-src` | Fallback for all fetch directives | `'self'` |
| `script-src` | Scripts (`'unsafe-inline'` would re-allow inline code — avoid; a *nonce* is a random one-time token) | `'self'` or `'nonce-r4nd0m'` |
| `script-src-attr` | Inline event-handler attributes (`onclick=…`) | falls back to `script-src` |
| `style-src`, `img-src`, `connect-src` (fetch/WebSocket — a persistent two-way connection), `font-src`, `frame-src` | Each resource type | `connect-src 'self' https://api.example.com` |
| `object-src 'none'` | Plugins (`<object>`, `<embed>`) | Always `'none'` |
| `base-uri 'self'` | Stops `<base>` tag hijacking relative URLs | |
| `frame-ancestors 'none'` | Who may embed you in an iframe — **clickjacking** (hiding your page in an invisible frame to trick clicks) protection (supersedes the older `X-Frame-Options` header) | |
| `report-to` / `report-uri` | Where to send violation reports | |

### Seen in a real browser

`code/csp.html` is served by the demo server with `Content-Security-Policy: default-src 'self'; script-src 'self'; img-src 'self'; style-src 'self' 'unsafe-inline'`. It contains an inline `<script>` that would replace a message, and loads `code/csp_app.js`, which injects an attacker's comment through `innerHTML` and logs `securitypolicyviolation` events:

```
// csp_app.js (allowed: same origin)
document.addEventListener('securitypolicyviolation', (e) => {
  log.textContent += `blocked ${e.effectiveDirective.padEnd(15)} ${e.blockedURI || '(inline)'}\n`;
});
document.getElementById('comment').innerHTML =
  'Great article! <img src="https://evil.example/pixel.png"
onerror="document.body.style.background=\'red\'">';
```

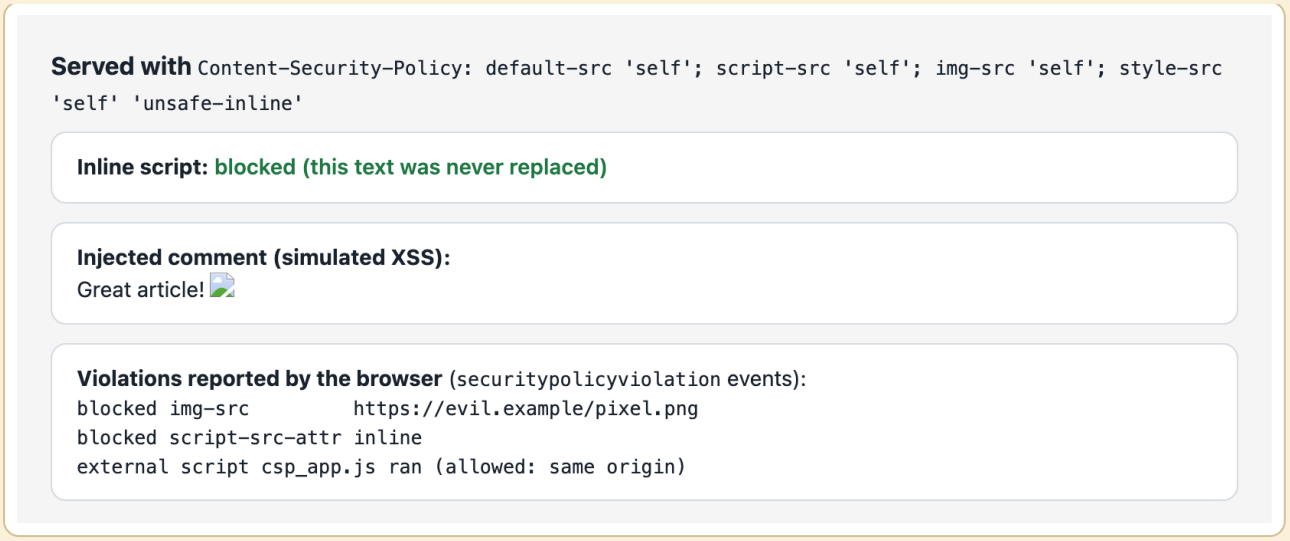



*Figure — the same kind of injection that took over the page in §14.10 is neutralized: the external image and the inline `onerror` handler were both blocked (headless Chrome).*

(The inline-script violation isn't in the log because it happened before `csp_app.js` registered its listener; it appears in the DevTools console.)

### How to adopt CSP

1. Move inline scripts and `on…=` attributes into external files and `addEventListener` (good practice anyway — Chapter 13 §13.5).
2. Start with `Content-Security-Policy-Report-Only` to collect violations without breaking the site.
3. For pages that need inline scripts (e.g. server-rendered data), use a per-response random **nonce** (`script-src 'nonce-…'`) — with `'strict-dynamic'` scripts loaded by trusted scripts are allowed too. This "strict CSP" is what Google recommends over long domain allow-lists, which are often bypassable.
4. Enforce, keep reporting, and review violations.

### Other security headers worth sending

| Header | Purpose |
|---|---|
| `Strict-Transport-Security: max-age=31536000; includeSubDomains` | HTTPS only (HSTS: HTTP Strict Transport Security) |
| `X-Content-Type-Options: nosniff` | Don't guess MIME types (labels like `text/html` naming a file's format; stops some script injection via uploads) |
| `Referrer-Policy: strict-origin-when-cross-origin` | Don't leak full URLs (the *referrer*, the page a request came from) to other sites (this is also the browser default today) |
| `Permissions-Policy: camera=(), geolocation=()` | Turn off powerful features you don't use |
| `Cross-Origin-Opener-Policy: same-origin` | Isolate your window from cross-origin popups |

### Security checklist for front-end code

| Risk | Rule |
|---|---|
| XSS | Data as text; escape by context; DOMPurify for HTML; allow-list URL protocols; strict CSP |
| Session theft | `HttpOnly; Secure; SameSite` cookies; never tokens in URLs |
| CSRF | SameSite + CSRF token or origin check; no state change on GET |
| Secrets | No API keys or secrets in front-end bundles |
| Dependencies | `npm audit` (scans installed packages for known vulnerabilities), lockfiles (files pinning exact dependency versions), few and well-maintained packages (Chapter 10); Subresource Integrity (`integrity=`, a hash the browser checks so a tampered file is refused) for third-party CDN (content delivery network) scripts |
| Clickjacking | `frame-ancestors 'none'` or `'self'` |
| Validation | Client-side validation is UX; the server validates everything |
| Prototype pollution (attacker data adding properties to `Object.prototype`, affecting every object) | Don't deep-merge untrusted JSON into objects (Chapter 9) |

> **TL;DR**
>
> CSP limits where scripts/styles/images may come from and blocks inline code, so injected markup can't execute. Roll out in report-only mode, prefer nonces + `'strict-dynamic'`, add HSTS, `nosniff` and `frame-ancestors`.

---

## Chapter 14 TL;DR

- **fetch**: resolves on any HTTP status (check `res.ok`), rejects on network failure/abort; read the body once; JSON needs `Content-Type` + `JSON.stringify`; time out with `AbortSignal.timeout`.
- **SOP & CORS**: origin = scheme + host + port. Pages can send anywhere but read only same-origin responses unless the server sends `Access-Control-Allow-Origin`. Non-simple requests get an `OPTIONS` preflight. Browsers enforce it.
- **Storage**: cookies (tiny, auto-sent; `HttpOnly; Secure; SameSite` for sessions), `localStorage` (persistent strings, ~5 MB, sync, never secrets), `sessionStorage` (per tab), IndexedDB (async database, structured data, large quota).
- **rAF** for visual updates: once per frame, time-based progress, paused in background tabs.
- **URL/URLSearchParams** for parsing, building, encoding and validating URLs.
- **History API**: `pushState`/`replaceState` + `popstate` = SPA routing; server fallback to `index.html`.
- **Web Workers**: parallel threads without DOM; `postMessage` with structured clone; keep CPU-heavy work off the main thread.
- **Observers**: `IntersectionObserver` (visibility, lazy loading), `ResizeObserver` (element size).
- **XSS**: data as text, context-aware escaping, URL allow-lists, DOMPurify, CSP, HttpOnly.
- **CSRF**: SameSite cookies + CSRF tokens/origin checks; no state changes on GET; CORS is not a CSRF defence.
- **CSP** and security headers are defence in depth.

Next: JavaScript on the server — Chapter 15: Node.js Basics.